

```
document.write("Searching: [" +  
haystack.slice(left, right + 1).join(", ") + "]<br />");  
  
if (left > right) { // BASE CASE  
    return null; // The `needle` is not in `haystack`.  
}  
  
let mid = Math.floor((left + right) / 2);  
if (needle == haystack[mid]) { // BASE CASE  
    return mid; // The `needle` has been found in `haystack`.  
} else if (needle < haystack[mid]) { // RECURSIVE CASE  
    return binarySearch(needle, haystack, left, mid - 1);  
} else if (needle > haystack[mid]) { // RECURSIVE CASE  
    return binarySearch(needle, haystack, mid + 1, right);  
}  
}  
  
document.write(binarySearch(13, [1, 4, 8, 11, 13, 16, 19, 19]));  
</script>
```

When you run these programs, the list [1, 4, 8, 11, 13, 16, 19, 19] is searched for 13, and the output looks like this:

```
Searching: [1, 4, 8, 11, 13, 16, 19, 19]  
Searching: [13, 16, 19, 19]  
Searching: [13]  
4
```

The target value 13 is indeed at index 4 in that list.

The code calculates the middle index (stored in `mid`) of the range defined by the `left` and `right` indices. At first, this range is the entire length of the items list. If the value at the `mid` index is the same as `needle`, then `mid` is returned. Otherwise, we need to figure out whether our target value is in the left half of the range (in which case, the new range to search is `left` to `mid - 1`) or in the right half (in which case, the new range to search is `mid + 1` to `end`).

We already have a function that can search this new range: `binarySearch()` itself! A recursive call is made on the new range. If we ever get to the point where the right end of the search range comes before the left, we know that